

AI / Data Science

Task Report — Day 04

Heart Disease Dataset · Model Comparison · Feature Selection · Tuning

DATE

15 June 2026

DOMAIN

Artificial Intelligence / Data Science

DATASET

Heart Disease Dataset (UCI) (303 rows x 14 cols)

1 Task Overview

#	Objective	Status
1	Train and compare multiple machine learning models	✓ Done
2	Perform feature selection on a dataset	✓ Done
3	Improve model accuracy through parameter tuning	✓ Done
4	Create visual reports and insights	✓ Done
5	Prepare project documentation	✓ Done

2 Dataset — Heart Disease (UCI)

Source: UCI Heart Disease Dataset **Rows:** 303 **Columns:** 14 **Task:** Binary Classification — predict presence of heart disease

Column	Description	Type
age	Age in years	Numeric
sex	1 = male, 0 = female	Categorical
cp	Chest pain type (0-3)	Categorical
trestbps	Resting blood pressure	Numeric
chol	Serum cholesterol	Numeric
fbs	Fasting blood sugar > 120 (1/0)	Categorical
restecg	Resting ECG result (0-2)	Categorical
thalach	Max heart rate achieved	Numeric
exang	Exercise-induced angina (1/0)	Categorical
oldpeak	ST depression from exercise	Numeric
slope	Slope of ST segment (0-2)	Categorical
ca	Number of major vessels (0-3)	Numeric
thal	Thalassemia type (1-3)	Categorical
target	1 = Heart Disease, 0 = No Disease (Target)	—

This dataset was already fully numeric with no missing values, so no cleaning was required. This allowed the focus to be entirely on model comparison, feature selection, and optimization.

3 Exploratory Data Analysis

Before training any models, the target distribution and feature correlations were examined. The dataset is fairly balanced between patients with and without heart disease, and several features (cp, thalach, exang, oldpeak) show clear correlation with the target variable.

4 Model Comparison (5-Fold Cross Validation)

Three classification models were trained and evaluated using both a single train/test split (Test Accuracy) and 5-fold cross validation (CV Mean Accuracy), which gives a more reliable performance estimate by averaging results across 5 different data splits. Features were scaled using StandardScaler before training Logistic Regression.

Model	Test Accuracy	CV Mean Accuracy
Logistic Regression	0.80	0.83
Decision Tree	0.70	0.76
Random Forest	0.84	0.84

```
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(random_state=42)
}

for name, model in models.items():
    model.fit(X_train, y_train)
    test_acc = accuracy_score(y_test, model.predict(X_test))
    cv_scores = cross_val_score(model, X, y, cv=5)
```

Insight Random Forest performed best overall (0.84 test accuracy, 0.84 CV accuracy). Logistic Regression was close behind (0.83 CV accuracy) despite being a much simpler model. Decision Tree performed worst (0.76 CV accuracy), likely overfitting on the full feature set without any depth restriction.

5 Visualizations

Chart 1

Target Distribution (Bar Chart) → chart1_target_distribution.png

Insight

The dataset is fairly balanced between patients with heart disease and those without, making it suitable for classification without needing to handle class imbalance.

Chart 2

Correlation Heatmap (Heatmap) → chart2_correlation_heatmap.png

Insight

cp (chest pain type), thalach (max heart rate), exang, and oldpeak show the strongest correlation with the target variable.

Chart 3

Model Comparison (Bar Chart) → chart3_model_comparison.png

Insight

Random Forest and Logistic Regression achieved similar cross-validation accuracy (~0.83-0.84), while Decision Tree lagged behind at 0.76.

Chart 4

Feature Importance (Horizontal Bar Chart) → chart4_feature_importance.png

Insight

cp (chest pain type) is the single most important feature (0.157), followed by thalach (0.117) and oldpeak (0.113).

Chart 5

Before vs After Tuning (Bar Chart) → chart5_before_after_tuning.png

Note

Surprisingly, the tuned Random Forest on selected features (0.79) performed worse than the default Random Forest on all features (0.84).

6 Feature Selection & Hyperparameter Tuning

Feature Selection

Using the Random Forest's built-in feature importance scores, the top 6 most predictive features were identified and selected for the tuning stage.

Rank	Feature	Importance Score
1	cp (chest pain type)	0.157
2	thalach (max heart rate)	0.117
3	oldpeak (ST depression)	0.113
4	thal (thalassemia)	0.107
5	chol (cholesterol)	0.087
6	ca (major vessels)	0.086

Hyperparameter Tuning (GridSearchCV)

GridSearchCV was used to search for the best Random Forest configuration (n_estimators, max_depth, min_samples_split) using 5-fold cross validation on the top 6 selected features.

```
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10],
    'min_samples_split': [2, 5]
}
grid = GridSearchCV(RandomForestClassifier(random_state=42),
                    param_grid, cv=5, scoring='accuracy')
grid.fit(X_train_sel, y_train)
```

Configuration	Accuracy
Default Random Forest (all 13 features)	0.84
Tuned Random Forest (top 6 features, best params: max_depth=5, min_samples_split=5, n_estimators=50)	0.79

Note Reducing to 6 features and tuning the model actually **lowered** accuracy from 0.84 to 0.79. This shows that feature selection and hyperparameter tuning do not always improve performance — on this dataset, the full feature set with default settings generalised better.

7 Key Findings & Conclusion

- Random Forest was the best overall model (0.84 test accuracy, 0.84 CV accuracy)
- Logistic Regression performed nearly as well (0.83 CV accuracy) despite being far simpler
- Decision Tree alone performed worst (0.76 CV accuracy), likely due to overfitting

- cp (chest pain type) is the strongest single predictor of heart disease
- Feature selection + tuning reduced accuracy (0.84 -> 0.79) on this dataset
- More features and default settings generalised better than a reduced, tuned model here

Task Status: Complete — All 5 objectives met. Three models trained and compared with cross validation, feature importance analysed and top features selected, hyperparameter tuning performed with GridSearchCV, 5-chart visual report created, and findings documented. Project structured and pushed to GitHub.